Jack Millington

Assignment 2 - Programming

2802ICT Intelligent Systems

S5405915

Table of Contents

Part 1 ..... ***3***

Software Design **3**

Test Results and Diagrams **5**

Discussion, Observation, and Insights of results. **7**

Conclusion **7**

Part 2 ..... ***8***

Software Design **8**

Experiments **9**

Discussion of Results: **13**

Conclusion: **14**

Appendix 1 ..... ***15***

Appendix 2 ..... ***23***

Figure 1: Training/ test set size and total accuracy 5
Figure 2: Counts per class in Train/ test set 5
Figure 3: Precision, Recall, and F1 Score metrics 5
Figure 4: ID3 Decision Tree Learning Curve 6
Figure 5: Experiment 1: Accuracy vs Epochs (epochs=30, batch=20, learning rate= 3.0) 9
Figure 6: Experiment 2: Accuracy vs Epochs for Different learning rates (epochs=30, batch=20) 10
Figure 7: Max Test Acc % for each learning curve 10
Figure 8: Experiment 3: Batch Size vs Accuracy (learning rate 3.0, epochs=30) 11
Figure 9: Max Test Acc % For Each Test Size 11
Figure 10: Experiment 4: Extended training (batch size= 100, learning rate=1.0, epochs=100) 12

Part 1

Software Design

The script reads the car.csv file and separates each row into a "*desc"* dictionary indicating the description of the car, this is appended into a list of descs. It then appends the class into a "ratings" list. Each value *i* in descs and ratings map to each other. These are then randomly shuffled and split into a training and test set. 80% of the data was chosen for the training set, and 20% was used as the test set.

A helper function count_ratings(ratings), returns a dict with the count of each rating in a list of ratings.

The function entropy(ratings), calculates the entropy of a list of ratings, this is used to calculate the information gain of an attribute.

The partition(descs, ratings, attribute) function, partitions a list of descs and ratings according to a specified attribute, where it returns a dict {attribute value: (list of desc with that attribute, list of ratings for those descs), …}

The weighted_entropy() function calculates the weighted entropy of an attribute. The information_gain() function calculates information gain of an attribute, this is the entropy of the ratings list minus the weighted entropy of an attribute. This is used to determine the attribute with the highest information gain in the ID3 algorithm.

The node class represents a node in the tree used in the ID3 algorithm, where a node can hold an attribute value, a dict of children, a majority_at_node value, and, if it is a leaf node, a prediction value. The methods of the class are is_leaf(), returning a Boolean value whether or not it's a leaf node, add_child(), which adds a child to its children dict, get_child() which returns a child node with "value", and set_majority, which sets the majority_at_node variable.

Two helper functions for building a tree are all_identical(), which returns whether all values in a ratings list are identical, and find_majority_label(), which returns the majority label on a subset of ratings. The global majority is also calculated in case of the need to fallback to a prediction value.

The build tree() function, is the main ID3 learning algorithm, and builds a decision learning tree from the set of training data. First it confirms inputs align, then computes the 3 base cases. Base case A returns the global majority if descs is empty. Base case B returns the node prediction if all of a subset of ratings are identical, and base case C returns the majority label in a subset of ratings if the attributes value is none. Then it finds the current best attribute by calculating the one with the highest information gain. It then chooses this attribute, and partitions by it. Checks whether or not it is a leaf node, if not it recursively splits further using the remaining attributes, and builds the tree.
The predict_one() function uses the test data and walks the tree until it reaches a leaf

node, and returns the predicted value. Predict_many() runs predict_one for all descs in the test set.

The training and test set sizes are then printed, the total accuracy is then calculated and printed. The per class metrics are printed, including precision, recall, and f1-score, and then the macro and weighted averages for these values were also printed for results analysis. Finally the learning curve was plotted, where the training set increased in increments of 10%, starting at 10% and finishing with the whole training set.

Test Results and Diagrams

The dataset the experiments were conducted with was car.csv. It has 1728 rows, 6 categorical features, and 4 classes. The training and test set was randomly split 80/20.

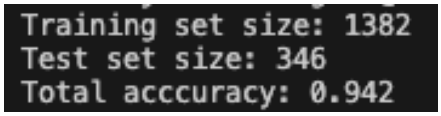

Figure 1: Training/ test set size and total accuracy

As seen in the above image, there was a total of 1382 values in the training set and 346 in the test set. When using the full training set, the decision learning tree achieved a 94.2% accuracy.

Below shows the counts per class for the training and test set.

Class	unacc	acc	good	vgood
Train	954	320	58	50
Test	256	64	11	15

Figure 2: Counts per class in Train/ test set

Below shows the per class metrics and the macro and weighted metrics. Precision shows "of what was predicted positive, how many were right". Recall shows "of all the true positives, how many were caught". F1 score is the harmonic mean of precision and recall that balances both.

Label	Precision	Recall	F1-Score
Class: unacc	0.980	0.977	0.978
Class: acc	0.873	0.859	0.866
Class: good	0.727	0.727	0.727
Class: vgood	0.765	0.867	0.812
Macro	0.836	0.857	0.846
Weighted	0.943	0.942	0.943

Figure 3: Precision, Recall, and F1 Score metrics

Below shows the learning curve of the data when tested in increments of 10% of the full training set.

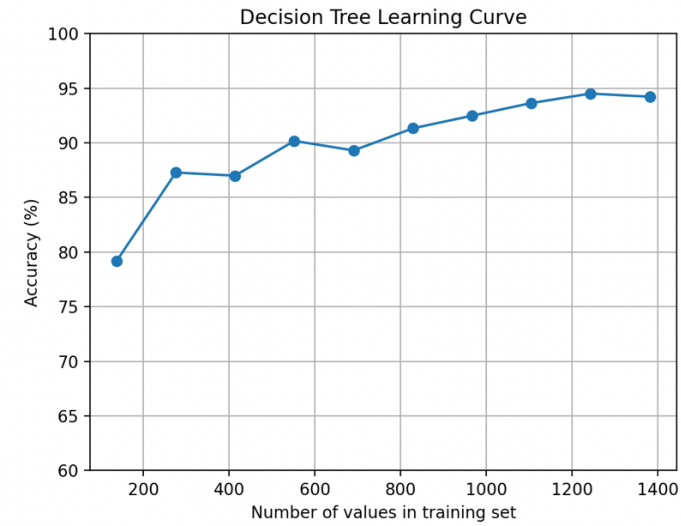

Figure 4: ID3 Decision Tree Learning Curve

Discussion, Observation, and Insights of results.

Overall, the model scores high weighted F1 at 0.943, but lower macro F1 at 0.846, this means there is strong headline accuracy with weaker consistency across classes. “unacc is near ceiling at 0.98 F1, which shows clear separability on attributes like safety or persons, reflecting the large amount of data with this class, (954 train/ 256 test). “acc” is solid at 0.866 F1, yet shows boundary confusions with neighbouring categories. “good” is weakest at 0.727 F1, indicating both over and under prediction, this is not surprising as there is scarce support due to only 58 train and 11 test values, while also overlapping with acc and vgood. Vgood has higher recall but lower precision, this suggests that slightly over general leaves admit non-vgood cases. It practice, this model should show great rejection detection, due to the accuracy of unacc, acceptable performance on acc, and noisier distinctions between good and vgood.

The curve shows test accuracy improving from roughly 78 percent when tested on 10% of the training data, about 100 training values, to approximately 94% when surpassing 1200 training values. The trajectory is monotonic overall with a small wobble around the mid range, then a clear diminishing returns plateau once above ~1000 samples. This pattern is typical of high bias learners as initially there is insufficient data to select informative splits, before transitioning into a variance limited regime since the tree captures the dominant structure of the data.
This learning curve demonstrates that the decision tree benefits strongly from additional data up to one thousand samples before performance plateaus around ninety five percent. This shape supports the conclusion that higher accuracy gains would likely come from changes such as pruning or engineered features and not merely more data

Conclusion

The ID3 decision tree achieved a 94.2% test accuracy with a weighted F1 of 0.943 and a lower macro F1 of 0.846. This indicates a strong overall performance but uneven class wise generalisation. Per class results show the majority classes have higher F1 scores, while the minority classes like good and vgood struggling in comparison. This shows that there is scarce minority support. The learning curve rises steeply with data the plateaus around ~1000 training samples at around 95%. This suggests diminishing returns from additional data under this hypothesis class. Overall, this model is highly reliable for rejecting unsuitable cars, and is generally competent at selecting acceptable cars. However decisions between good and very good cars remains noisier.
Key limitations of this experiment are no pruning / depth control, and class imbalance as there is many more unacceptable cars in the dataset compared to good or very good cars. This can inflate headline metrics and add variance to minority class scores. As results can vary across different random seeds, a future improvement could be to report the mean results over multiple seeds, to gather a more complete and accurate assessment of the data.

Part 2

Software Design

Part 2 of this assignment implements a 3 layer neural network, which trains on a fashion-mnist.csv.gz dataset. This data consists of 60000 training examples and 10000 tests examples. Each example is a greyscale image associated with a label from 10 classes. Each image is 28x28 Pixels, where each pixel has a greyscale value from 0-255. The datasets have 785 columns, with the first being the label, and the next 784 being the pixel values.

The program starts with the user parsing in a command of the format:
"python nn.py NInput NHidden NOutput train.csv.gz test.csv.gz"
Where NInput, NHidden, and NOutput are the number of neurons on each layer of the network. In this experiment, we are using 784, 30, 10 respectively, as there are 784 pixels, and 10 output labels. There are also 30 neurons in the hidden layer.

The data is loaded and the x and y values are normalised. The x values are the pixel values normalised from 0:255 to 0:1, to coincide with the math later on. The Y values change the single value, e.g '3', to an array [0,0,0,1,0,0,0,0,0,0], to represent the output layer.

The data is split into batches, to lower the memory load, where the batch size can be set, and these batches are transposed, from x = (N, 784), y = (N, 10) to (784, N) and (10, N) to make it better for the math formulas.

The weights and biases are initialised. Where the weights are randomly chosen in uniform distribution to the Xavier/ Glorot scale. The biases are all initialised to zero.

To train the neural network, the forward propagation algorithm is used, and then the back propagation algorithm. Over many iterations, through training examples, this manipulates the individual weights and biases of each neuron and connection to be able to predict the label of the 28x28 image. One epoch is one full run of the training data.

Task 1 then plots the accuracy vs epochs over time with a batch size of 20, a learning rate of 3.0, and across 30 epochs.

Task 2 then plots multiple curves on top of each other that have various changed learning rates to extrapolate the results when learning rate is changed.

Task 3 plots batch size vs accuracy percentage for varying batch sizes.

Experiments

Experiment 1: Accuracy vs Epochs (epochs=30, batch=20, learning rate= 3.0)

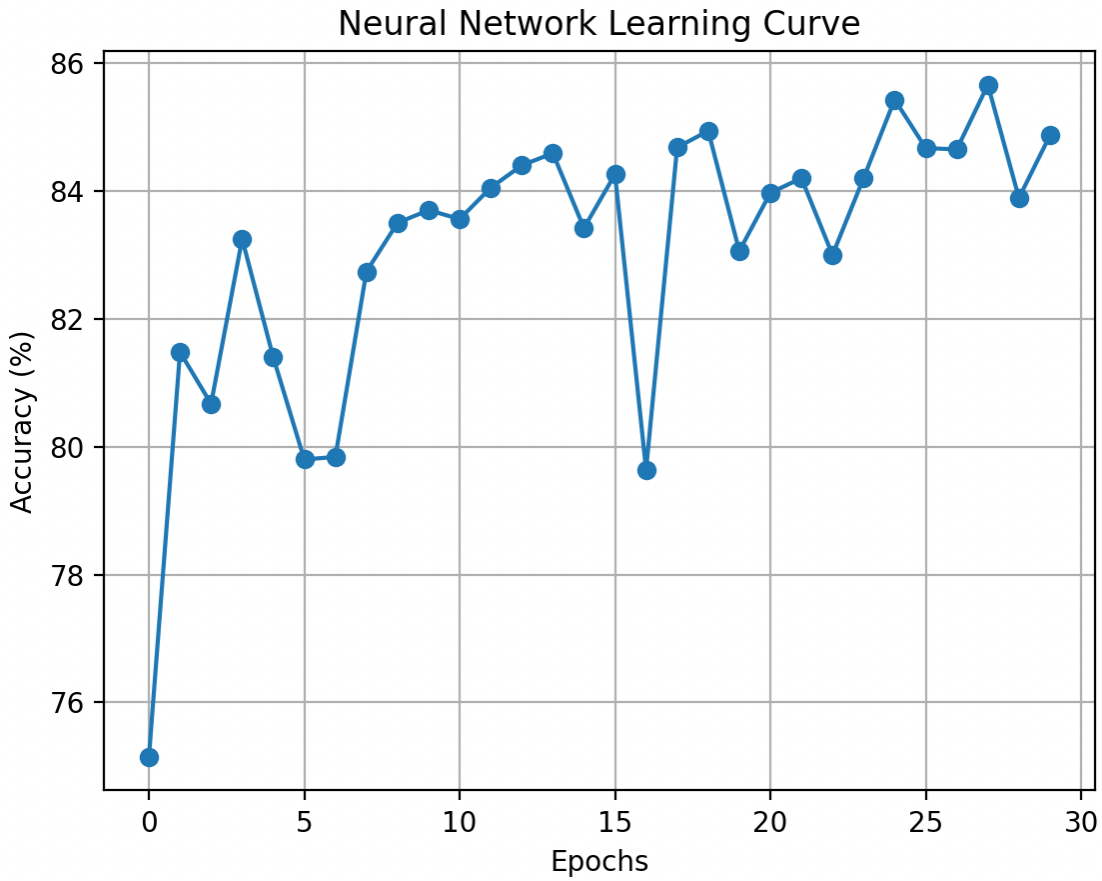

Figure 5: Experiment 1: Accuracy vs Epochs (epochs=30, batch=20, learning rate= 3.0)

Max Test Accuracy = 85.66%

Experiment 2: Accuracy vs Epochs for Different learning rates (epochs=30, batch=20).

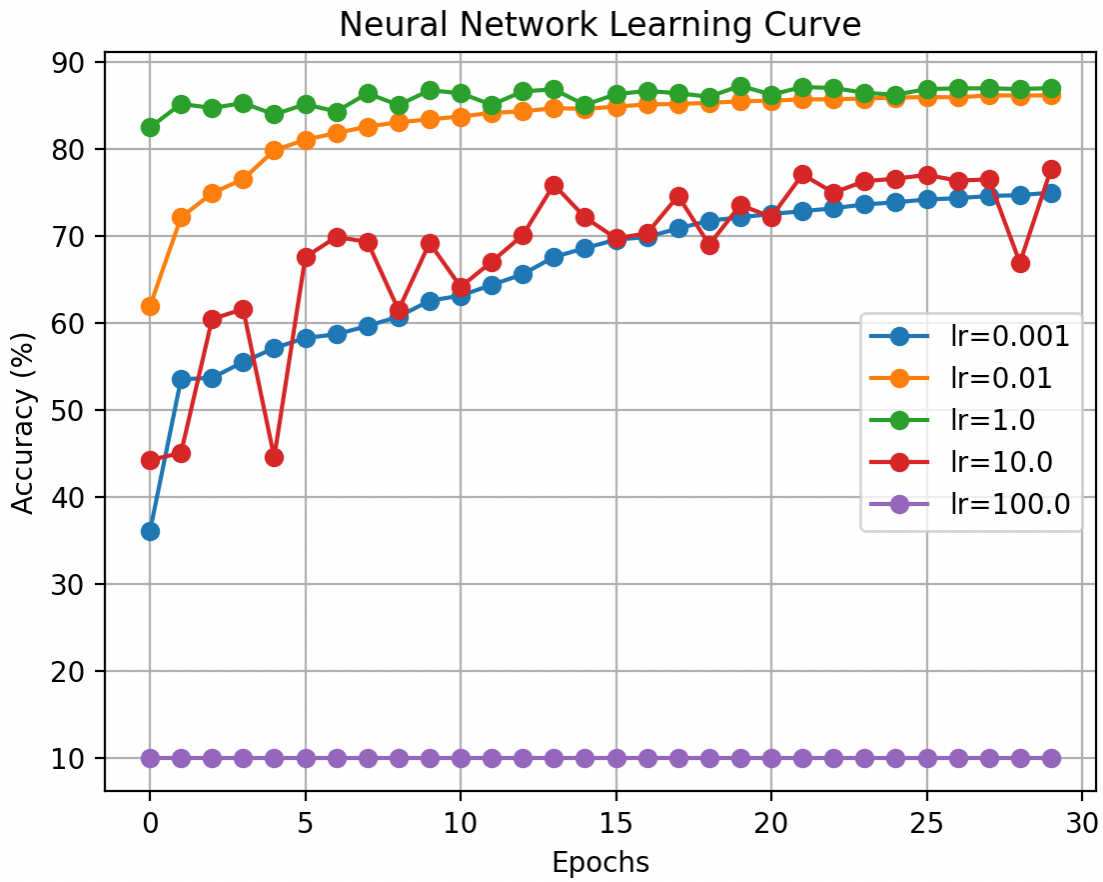

Figure 6: Experiment 2: Accuracy vs Epochs for Different learning rates (epochs=30, batch=20).

η	Max Test Accuracy %
0.001	74.94%
0.01	86.12%
1.0	87.22%
10.0	77.62%
100.0	10.00%

Figure 7: Max Test Acc % for each learning curve

Experiment 3: Batch Size vs Accuracy (learning rate 3.0, epochs=30)

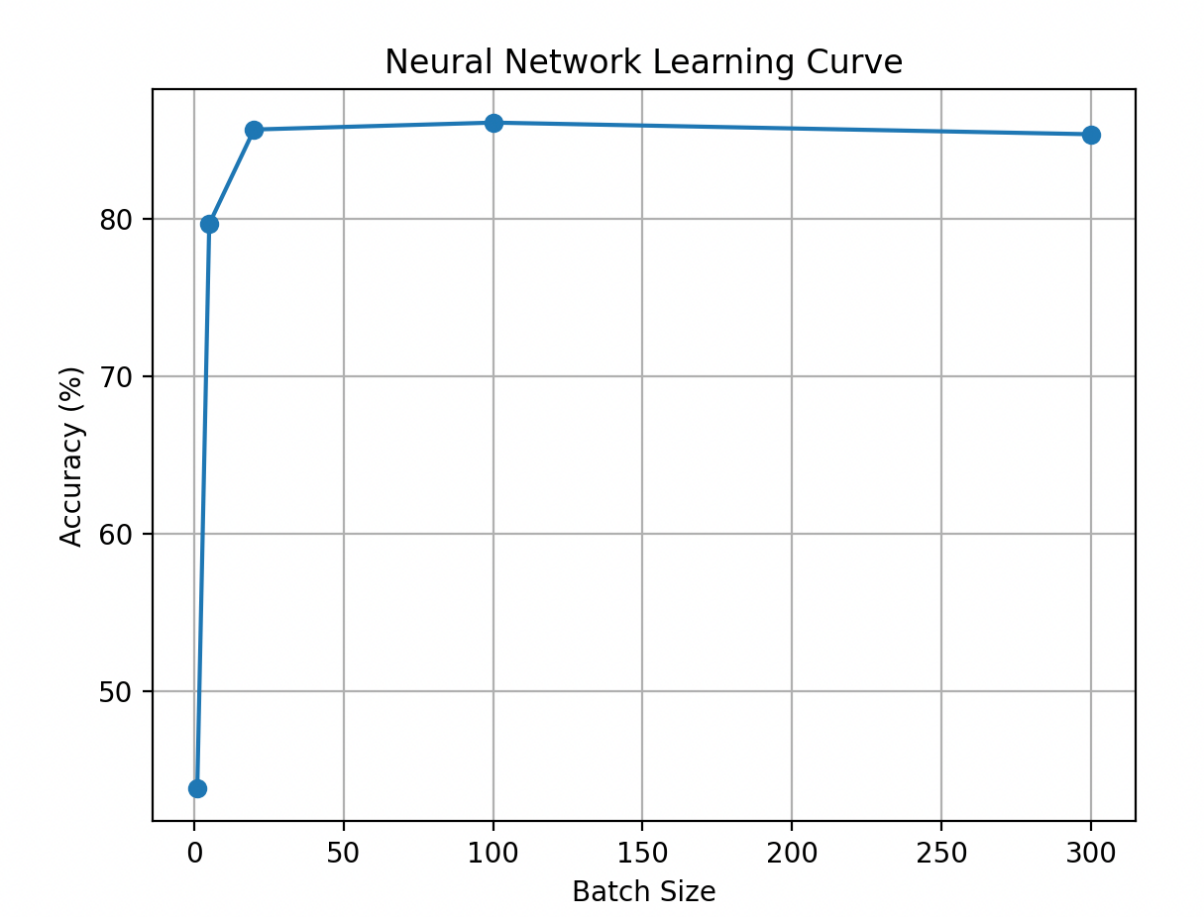

Figure 8: Experiment 3: Batch Size vs Accuracy (learning rate 3.0, epochs=30)

Batch Size	Max test accuracy %
1	43.86%
5	79.67%
20	85.66%
100	86.10%
300	85.36%

Figure 9: Max Test Acc % For Each Test Size

Experiment 4: Extended training (batch size= 100, learning rate=1.0, epochs=100)

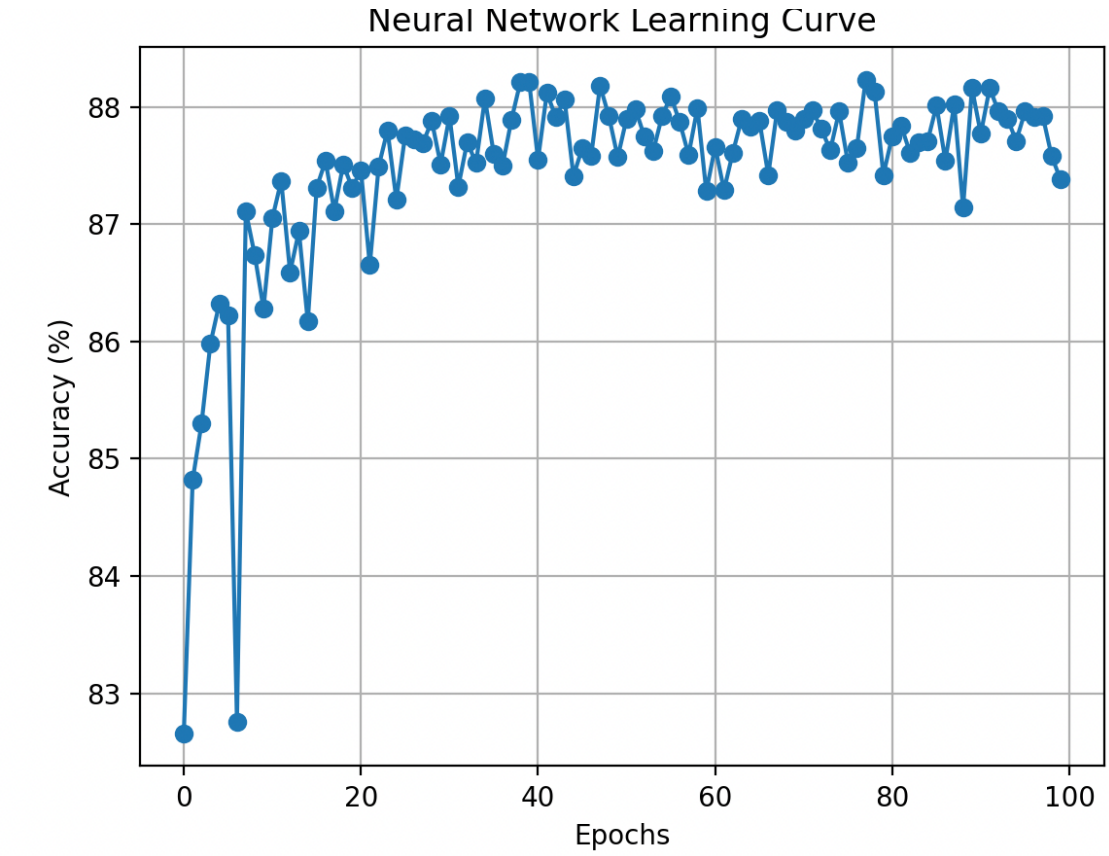

Figure 10: Experiment 4: Extended training (batch size= 100, learning rate=1.0, epochs=100)

Max Test Accuracy = 88.23%

Discussion of Results:

Experiment 1: Accuracy vs Epochs (epochs=30, batch=20, learning rate= 3.0)

Accuracy climbs from ~75-80% in early epochs to a best of 85.66% by the end. The upward trend with small oscillations is expected as the early epochs reduce bias quickly, then progress slows as updates bounce around narrow minima. To late epoch collapse is visible which indicates that overfitting is limited at this depth.

Experiment 2: Accuracy vs Epochs for Different learning rates (epochs=30, batch=20).

The learning rate sizes tested were η =0.001, 0.01, 1.0, 10, 100. The learning rate of 1.0 achieved the highest accuracy of 0.8722. The learning rate controls the step size. A very small learning rate underfits within 30 epochs as it has slow convergence. η=1.0 hits the best trade off as it achieved fast progress and does not destabilise the sigmoid regime. Large learning rates like 10.0 oscillate around good regions while 100.0 effectively fails. This may be because it causes gradients to explode and stops it from learning altogether. This proves that the learning rate is the dominant lever for this model.

Experiment 3: Batch Size vs Accuracy (learning rate 3.0, epochs=30)

With epochs fixed at 30, the very small batch sizes make extremely noisy updates, casing the model to settle within the time budget, therefore batch=1 collapses. Moderate to large size batches between 20 and 100 give the best results by reducing gradient variance while still providing enough update steps per epoch. Very large batches like 300 lower the update count and can underfit unless epochs or learning rate is increased.

Experiment 4: Extended training (batch size= 100, learning rate=1.0, epochs=100)

The batch size and learning rate were chosen as these values as they performed with the highest accuracy in the previous tests, so it was logical to use them both at the same time to try and gain a higher accuracy. 100 epochs were chosen to confirm that the curve had plateaued, and that significant increase in accuracy was not possible. The test accuracy rises rapidly in the first 5 to 8 epochs and crosses 87 percent by 10 epochs, reaching a peak of 0.8823 around the 30 to 50 epoch window, and then proceeds to narrowly fluctuate until 100 epochs with no late collapse. The gain vs results in experiment 1 show a ~2.6% increase in accuracy. This comes from the better learning rate and batch pairing found in earlier experiments and giving the model enough epochs to finish convergence. Since the model plateaued around 50 epochs, a budgeted run with 40-50 epochs with a learning rate of 1.0 and a batch size of 100 is a strong default.

Overall: The best setting found was (batch size= 100, learning rate=1.0, epochs=100) which peaked at 88.23% accuracy. It was found that the learning rate matters most, followed by the batch size, and then the number of epochs (after ~40). It was found that the model would likely fail if it had too high of a learning rate with it destabilising at 10.0 and collapsing at 100.0. Too small of a batch size wastes epochs on noisy steps, while too large batch sizes may require more epochs or a slightly higher learning rate.

Conclusion:

The neural network reached a best test accuracy of 88.23% using a learning rate of 1.0 and a batch size of 100, with performance peaking around 30 to 50 epochs and the plateauing. The experiments show that the learning rate is the primary driver of outcomes, where the network begins to destabilise at 10.0, and collapse at 100.0. It was found that moderate to large batches between 20 and 100 outperform very small or large batches such as 1 or 1000. The improved hyper parameter settings made in experiment 4 improved the accuracy by 2.6% compared to the initial experiment 1. Key limitations include the fixed [784, 30, 10] architecture with sigmoid units which are susceptible to saturation, and no explicit regularisation. Overall, the experiments demonstrate clear, well-reasoned hyperparameter effects and a justified configuration that achieves strong generalisation on Fashion-MNIST. As results can vary across different random seeds, a future improvement could be to report the mean results over multiple seeds, to gather a more complete and accurate assessment of the data.

Appendix 1

```
import csv
import random
import math
import matplotlib.pyplot as plt
import numpy as np

#  --- Read CSV ---
descs, ratings = [], []
with open("car.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f) # uses the header row as keys
    for row in reader:
        ratings.append(row["class"])
        descs.append({
            "buying": row["buying"],
            "maint": row["maint"],
            "doors": row["doors"],
            "persons": row["persons"],
            "lug_boot": row["lug_boot"],
            "safety": row["safety"],
            })
attributes = ["buying", "maint", "doors", "persons", "lug_boot", "safety"]
classes = ["unacc", "acc", "good", "vgood"]
# descs is a list of dics, e.g [{'buying': 'vhigh', 'maint': 'vhigh', 'doors':
'2', 'persons': '2', 'lug_boot': 'small', 'safety': 'low'}, ...]
# ratings is a list of strins, e.g. ['unacc', 'unacc', 'good', ...]
# attributes is a list of all attributes of the car, except class, as this is the
rating

# --- Random shuffle of data ---
data = list(zip(descs, ratings)) # pair descs with thier rating for random shuffle

random.seed(67) # set shuffle seed for reproducibility
random.shuffle(data)

# unzip back to desc and rating
descs, ratings = zip(*data) # turns into tuples
descs, ratings = list(descs), list(ratings) # converts back to lists

# --- Split data into train and test sets
splitRatio = 0.8 # 80% of the data is training and 20% is testing
change = int(splitRatio * len(ratings))
train_d, train_r, test_d, test_r = [],[],[],[]
for i in range(len(ratings)):
    if i < change:
        train_d.append(descs[i])
        train_r.append(ratings[i])
    else:
```

```
        test_d.append(descs[i])
        test_r.append(ratings[i])

# --- rating counts
# returns count of each rating e.g. {'unacc': 3, 'acc': 2, 'good': 1}
def count_ratings(ratings):
    counts = {}
    for rating in ratings:
        if rating not in counts:
            counts[rating] = 0
        counts[rating] += 1
    return counts

# --- calculates entropy of ratings - lower = higher confidence
def entropy(ratings):
    counts = count_ratings(ratings)
    total = len(ratings)
    ent = 0.0
    for rating, count in counts.items():
        p = count / total
        calc = -p * math.log(p, 2)
        ent += calc
    return ent

# --- Partition attribute
def partition(descs, ratings, attribute):
    partitions = {} # initialise an empty dict

    for desc, rating in zip(descs, ratings): # loop through each row in dataset
        value = desc[attribute] # e.g. "low", "med", "high"

        # if value not seen, make new entry
        if value not in partitions:
            partitions[value] = ([],[]) # start with 2 empty lists

        # Add current row to right place
        desc_list, rating_list = partitions[value]
        desc_list.append(desc)
        rating_list.append(rating)

    return partitions
# Returns a dict: {attribute value: (list of desc with that attribute, list of
ratings)}

# --- Weighted Entropy
# Returns the weighted entropy of an attribute to determine its information gain
def weighted_entropy(descs, ratings, attribute):
    if not descs or not ratings:
        return 0.0
    result = 0.0
```

```
    for value, (desc_list, rating_list) in partition(descs, ratings,
attribute).items():
        value_size = len(desc_list) # number of instances where attribute label
exists
        value_entropy = entropy(rating_list) # count of each rating
        weight = value_size / len(descs) # weight of value is value frequency /
total dataset
        result += weight * value_entropy
    return result

# Returns the information gain of an attribute
def information_gain(descs, ratings, attribute):
    return entropy(ratings) - weighted_entropy(descs, ratings, attribute)


# Tree data structure
class Node:
    def __init__(self):
        self.attribute = None # feature name used to split at this node
        self.children = {} # mapping from attribute value --> child node
        self.prediction = None # the class label if this node is a leaf
        self.majority_at_node = None # most common class among the rows that
reach this node
        ## A leaf has prediction set and attribute unset, vice versa for an
internal node

    def is_leaf(self): # Returns a boolean value whether it is a leaf node
        return self.prediction is not None

    def add_child(self, value, node): # adds a child node to the current node as
a dict entry
        self.children[value] = node

    def get_child(self, value): # retrieves a child node with "value"
        return self.children.get(value)

    def set_majority(self, label): # sets the majority at node variable to a
label of an attribute
        self.majority_at_node = label

# check if all values in ratings are identical
def all_identical(ratings):
    if not ratings:
        return False
    for label in ratings:
        if label != ratings[0]:
            return False
    return True

# returns majority label on a subset of ratings
def find_majority_label(ratings):
```

```
    counted_ratings = count_ratings(ratings)
    majority_label = max(counted_ratings, key=counted_ratings.get)
    return majority_label

# Global majority label used as default
global global_majority_rating
global_majority_rating = find_majority_label(train_r)

def build_tree(descs, ratings, attributes):
    # confim inputs align
    if not len(descs) == len(ratings):
        raise IndexError("Descs and ratings not the same")

    node = Node()
    # base case A, if descs is empty, return a leaf node with prediction as a
sensible default
    if not descs:
        node.prediction = global_majority_rating
        return node

    # base case B, check if all values in ratings are identical, and return that
rating if they are
    if all_identical(ratings):
        node.prediction = ratings[0]
        return node

    # base case C, check if attributes is empty, if so, return majority label in
ratings
    if not attributes:
        node.prediction = find_majority_label(ratings)
        return node

    # find best attribute
    best_attr = None
    best_ig = 0
    for a in attributes:
        ig = information_gain(descs, ratings, a)
        if ig >= best_ig:
            best_ig = ig
            best_attr = a
    ### TO DO maybe add a rule where if best_ig or the subset is too small,
return leaf node with prediction = majority label

    # if best_attr is still none, then every ig is exactly 0, thus set it as the
majority label
    if best_attr is None or best_ig <= 0:
        node.prediction = find_majority_label(ratings)
        return node

    # set node attribute to best attribute and set majority at node
    node.attribute = best_attr
```

```
    node.majority_at_node = find_majority_label(ratings)

    # partition data by best attribute
    parts = partition(descs, ratings, best_attr)
    for value, (desc_list, rating_list) in parts.items():
        # if ratings list is empty, create a leaf node with prediction as the
majority rating at that attribute and attach to tree as a child of that attribute
        if not rating_list:
            child_node = Node()
            child_node.prediction = node.majority_at_node
            node.add_child(value, child_node)
            continue
        # if ratings are all the same create a child leaf node with the
prediction as that rating
        if all_identical(rating_list):
            child_node = Node()
            child_node.prediction = rating_list[0]
            node.add_child(value, child_node)
            continue
        # Otherwise, split futher with all attributes execpt the best attribute
        remaining_attributes = []
        for attr in attributes:
            if attr != best_attr:
                remaining_attributes.append(attr)
        child_node = build_tree(desc_list, rating_list, remaining_attributes)
        node.add_child(value, child_node)
    return node

# Predict the rating of a desc, by walking the tree that was built
# root is the root node of the tree, desc is a description of a car, returns a
rating
def predict_one(root: Node, desc: dict) -> str:
    node = root
    # return the prediction if we are at the leaf node, else move down tree
    while not node.is_leaf():
        # retrive attribute of node and find this attribute in desc
        attr = node.attribute

        # fallback if no attribute on non leaf node for some reason
        if attr is None:
            return node.majority_at_node

        value = desc.get(attr) # get value of attribute in desc
        child_node = node.get_child(value) # get child node
        if child_node is not None: # check if this value is in child branch
            # follow tree iteratively
            node = child_node
            continue
        else:
            # safety if value not in child branch
            return node.majority_at_node
```

```
    return node.prediction

# runs predictions for each desc and returns a list of predictions, can be mapped
with descs to determine results and accuracy
def predict_many(root: Node, descs: list[dict]) -> list:
    predictions = []
    for desc in descs:
        pred = predict_one(root, desc)
        predictions.append(pred)
    return predictions

### Testing

## Print Training and Test dataset sizes
print(f"Training set size: {len(train_d)}")
print(f"Test set size: {len(test_d)}")

## compute and print total accuracy
def calcAccuracy(predicted: list, actual: list):
    # check lists are same length
    if len(predicted) != len(actual):
        raise IndexError("predicted and actual are not same length")

    accuracy = 0.0
    correct_increase = 1/len(predicted)
    for i in range(len(predicted)):
        if predicted[i] == actual[i]:
            accuracy += correct_increase
    return accuracy

train_tree = build_tree(train_d, train_r, attributes)
test_results = predict_many(train_tree, test_d)
total_accuracy = calcAccuracy(test_results, test_r)
print(f"Total acccuracy: {total_accuracy:.3f}")

## Per class metrics
# all metrics is a dict: {class: [tp, fp, tn, fn, precision, recall, f1_score],
...}
all_metrics = {}
for c in classes:
    # initialise metrics for the class
    tp, fp, tn, fn = 0,0,0,0
    for i in range(len(test_results)):
        pred = test_results[i]
        truth = test_r[i]
        # true positive
        if pred == c and truth == c:
            tp += 1
        # false positive
        elif pred == c and truth != c:
            fp += 1
```

```
        # false negative
        elif pred != c and truth == c:
            fn += 1
        # true negative
        else:
            tn += 1
    # precision is how many predicted positives are actually correct, high = few
false alarms
    precision = tp / (tp + fp)
    # recall is how many real positives are caught, hig = few missed cases
    recall = tp / (tp + fn)
    # F1-score rewards balance between precision and recall
    f1_score = (2 * precision * recall) / (precision + recall)

    class_metric = [tp,fp,tn,fn, precision, recall, f1_score]
    all_metrics[c] = class_metric

# Macro averages for precision, recall and f1 score
macro_P = sum(all_metrics[c][4] for c in classes) / len(classes)
macro_R = sum(all_metrics[c][5] for c in classes) / len(classes)
macro_F = sum(all_metrics[c][6] for c in classes) / len(classes)

# Weighted averages for precision, recall and f1 score
def class_weight(c: list):
    return c[0] + c[3]
weighted_P = sum(all_metrics[c][4] * class_weight(all_metrics[c]) for c in
classes) / sum(class_weight(all_metrics[c]) for c in classes)
weighted_R = sum(all_metrics[c][5] * class_weight(all_metrics[c]) for c in
classes) / sum(class_weight(all_metrics[c]) for c in classes)
weighted_F = sum(all_metrics[c][6] * class_weight(all_metrics[c]) for c in
classes) / sum(class_weight(all_metrics[c]) for c in classes)

# Print frequency of each class
print("Train set class counts:", {c: train_r.count(c) for c in classes})
print("Test set class counts:", {c: test_r.count(c) for c in classes})

## Print metrics
for c in classes:
    print(f"Class: {c}, Precision: {all_metrics[c][4]:.3f}, Recall:
{all_metrics[c][5]:.3f}, F1-score: {all_metrics[c][6]:.3f}")
print(f"Macro Precision: {macro_P:.3f}, Macro Recall: {macro_R:.3f}, Macro F1-
Score: {macro_F:.3f}")
print(f"Weighted Precision: {weighted_P:.3f}, Weighted Recall: {weighted_R:.3f},
Weighted F1-Score: {weighted_F:.3f}")

## Plot learning Curve

# create x axis in sets of 10%:
x = [] # % of training seed
for i in range(1,11):
    x.append(i*10) # % of training seed
```

```
x_numOfTrainingSets = []
y = [] # accuracy 0-1
for percentage in x:
    train_subset_d = train_d[0:int(len(train_d) * (percentage/100))]
    train_subset_r = train_r[0:int(len(train_r) * (percentage/100))]
    train_tree = build_tree(train_subset_d, train_subset_r, attributes)
    test_results = predict_many(train_tree, test_d)
    total_accuracy = calcAccuracy(test_results, test_r)
    y.append(total_accuracy)
    x_numOfTrainingSets.append(len(train_subset_d))

y_percentage = [a*100 for a in y] # convert y to a percentage

plt.plot(x_numOfTrainingSets, y_percentage, marker='o') # line with circular
markers
plt.ylim(60, 100) # y axis to go from 0 to 100
plt.xlabel("Number of values in training set")
plt.ylabel("Accuracy (%)")
plt.title("Decision Tree Learning Curve")
plt.grid(True) # adds a grid
plt.show() # display window
```

Appendix 2

```
import numpy as np
import matplotlib.pyplot as plt
import argparse

def parse_args():
    p = argparse.ArgumentParser(
        description="python nn.py NInput NHidden NOutput train.csv.gz
test.csv.gz"
    )
    p.add_argument("NInput", type=int)
    p.add_argument("NHidden", type=int)
    p.add_argument("NOutput", type=int)
    p.add_argument("train_path")
    p.add_argument("test_path")
    return p.parse_args()

# Load the text
def loadFile(fName):
    R = np.loadtxt(fName, delimiter=',', dtype=np.int32, ndmin=2, skiprows=1)
    y_raw = R[:, 0] # collects all labels, 1 on each row
    x_raw = R[:, 1:785] # collects all 784 pixel values for each row
    return x_raw, y_raw

# Normalise the pixels to [0,1]
def normaliseX(x_raw):
    x = np.asarray(x_raw) # uses numpy array to decrease time cost as all happens
inside C
    return x / 255.0 # returns the array with each element divided by 255.0 to
normalise from 0 to 1

# changes [3,7,1,3,2] to [[0,0,0,1,0,0,0,0,0,0],[0,..],..]
def normaliseY(y_raw, num_classes=10):
    y = np.asarray(y_raw, dtype=np.int32) # convert y_raw to an array of int32
    I = np.eye(num_classes) # returns a 10x10 identity matrix
    return I[y] # picks row of matrix I for each value in y

def make_batches(N, batch_size, rng): # yields a list of indexes of batch size
    indicies = rng.permutation(N) # returns a random list of indicies with values
[0,N-1]
    for start in range(0,N,batch_size): # loops over a list where if batch size =
20, [0,20,40,60,..,N]
        yield indicies[start:start+batch_size] # returns the list of that batch,
stores where it is up to in loop and waits for next call to return the next batch

def transpose(x_rows, y_rows, indexes):
    # collect batches of only the specified indexes
    x_batch = x_rows[indexes]
    y_batch = y_rows[indexes]
```

```
    # transpose them from x = (N, 784), y = (N, 10) to (784, N) and (10, N) to
make it better for the math formulas
    x_trans = x_batch.T
    y_trans = y_batch.T
    return x_trans, y_trans

# initialises weights and bias for each neuron, input is number of neurons on
each layer ouptpt is a dict with all weights and biases
def init_weights_and_bias(n_in, n_hidden, n_out, rng=None):
    if rng is None:
        rng = np.random.default_rng(69)
    # Xavier/Glorot scale for sigmoid, good area to randomly initialse weights
    limit_W1 = np.sqrt(1/n_in) # weight scale for connection n_in --> n_hidden
    limit_W2 = np.sqrt(1/n_hidden) # weight scale for connection n_hidden -->
n_out
    # Random uniform distribution of weights using limits above to every
connection in neural network
    w1 = rng.uniform(-limit_W1, limit_W1, size=(n_hidden,
n_in)).astype(np.float32)
    w2 = rng.uniform(-limit_W2, limit_W2, size=(n_out,
n_hidden)).astype(np.float32)
    # Sets each hidden and output neuron bias to 0
    b1 = np.zeros((n_hidden, 1))
    b2 = np.zeros((n_out, 1))
    return {"w1":w1,"b1":b1,"w2":w2,"b2":b2}

def sigmoid(z):
    z = np.clip(z, -40.0, 40.0) # prevents exp overflow
    return 1 / (1+np.exp(-z)) # returns sigmoid values of Z array

def forward_prop(w_and_b, x_batch):
    # hidden pre-activation
    Z1 = w_and_b["w1"] @ x_batch + w_and_b["b1"] # @ is matrix multiplication *
is elementwise
    # hidden activation
    A1 = sigmoid(Z1)
    # output pre-activation
    Z2 = w_and_b["w2"] @ A1 + w_and_b["b2"]
    # output activation
    A2 = sigmoid(Z2)
    # predictions per sample
    pred = np.argmax(A2, axis=0)
    return {"z1":Z1, "z2": Z2, "a1": A1, "a2": A2, "pred": pred}

def back_prop(x_batch, y_batch, z_and_a, w_and_b):
    # output layer error
    s2 = z_and_a["a2"] - y_batch # simplified as using sigmoid + cross entropy
(shape is (10, m))
    # hidden layer error
    s1 = w_and_b["w2"].T @ s2 * z_and_a["a1"] * (1-z_and_a["a1"]) # z_and_a["a1"]
* (1-z_and_a["a1"]) is derivative of sigmoid, shape is (n_hidden, m)
```

```
    # output layer gradients
    m = y_batch.shape[1]
    dw2 = (1 / m) * s2 @ z_and_a["a1"].T # shape (10, NHidden)
    db2 = (1 / m) * np.sum(s2, axis=1, keepdims=True) # shape (NOutput, 1)
    # hidden layer gradients
    dw1 = (1 / m) * s1 @ x_batch.T # shape (NHidden, NInput)
    db1 = (1 / m) * np.sum(s1, axis=1, keepdims=True) # shape (NHidden, 1)
    return {"dw1": dw1, "db1" : db1, "dw2" : dw2, "db2" : db2}


def update_parameters(w_and_b, back_prop, learning_rate):
    w_and_b["w1"] -= learning_rate * back_prop["dw1"] # Updates W1 to reflect new
weights
    w_and_b["w2"] -= learning_rate * back_prop["dw2"] # Updates B1 to reflect new
Biases
    w_and_b["b1"] -= learning_rate * back_prop["db1"] # Updates W2 to reflect new
weights
    w_and_b["b2"] -= learning_rate * back_prop["db2"] # Updates B2 to reflect new
Biases
    return w_and_b

# Trains one epoch
def train_one_epoch(x_train, y_train, w_and_b, batch_size, learning_rate, epoch):
    total_loss = 0.0 # initialise loss to 0
    num_batches = 0 # initialise num batches to 0
    rng = np.random.default_rng(69+epoch) # rng is 69+epoch to have fresh seed
each run but ensure reproducability
    for indicies in make_batches(len(x_train), batch_size, rng):
        Xb, Yb = transpose(x_train, y_train, indicies) # transpose x and y batch
to match math formulas
        z_and_a = forward_prop(w_and_b, Xb) # runs forward propagation algorithm
        dw_and_db = back_prop(Xb, Yb, z_and_a, w_and_b) # runs back propagation
algorithm
        w_and_b = update_parameters(w_and_b, dw_and_db, learning_rate) # updates
the parameters
        num_batches += 1 # adds counter to num batches
    return w_and_b

def epoch_accuracy(x_test, y_test, w_and_b, batch_size): # cacluates accuracy of
one epoch
    num_correct = 0 # initialise num correct to 0
    for indicies in make_batches(len(x_test), batch_size,
rng=np.random.default_rng(69)):
        Xb, Yb = transpose(x_test, y_test, indicies) # transpose x and y batch to
match math formulas
        pred = forward_prop(w_and_b, Xb)["pred"] # runs forward propagation
algorithm to cacluate a prediction
        true = np.argmax(Yb, axis=0) # finds the true value in y
        num_correct += np.sum(pred == true) # checks if prediction == true and +1
to num correct if so
    return num_correct / y_test.shape[0] # returns accuracy from 0-1
```

```python
def train(x_train, y_train, x_test, y_test, w_and_b, batch_size, learning_rate, epochs): # trains neural network using specified hyper parameters
    accuracy_array = [] # initialises accuracy array to empty
    for i in range(epochs): # loops over number of epochs
        w_and_b = train_one_epoch(x_train, y_train, w_and_b, batch_size, learning_rate, epoch=i) # trains one epoch
        acc = epoch_accuracy(x_test, y_test, w_and_b, batch_size) # calculates acuracy after one is trained
        accuracy_array.append(acc) # append the accuracy to the list
    max_test_acc = np.max(accuracy_array) # finds the max test accuracy
    return max_test_acc, accuracy_array # returns the accuracy array and the max acc

def plot_task_1(accuracy_array, epochs):
    ## Plot learning Curve
    # create x axis based on epochs:
    x = [i for i in range(epochs)]
    y = [i*100 for i in accuracy_array] # accuracy % 0-100
    plt.plot(x, y, marker='o') # line with circular markers
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy (%)")
    plt.title("Neural Network Learning Curve")
    plt.grid(True) # adds a grid
    plt.show() # display window

def task1(xtr, ytr, xte, yte, params, batch_size, learning_rate, epochs): # runs and plots experiment 1
    max_test_acc, accuracy_array = train(xtr, ytr, xte, yte, params, batch_size, learning_rate, epochs)
    plot_task_1(accuracy_array, epochs)
    print(f"Task 1 Max test accuracy: {max_test_acc}")

def task2(xtr, ytr, xte, yte, NIn, NHid, NOut, batch_size, epochs): # runs and plots experiment 2
    learning_rates = [0.001, 0.01, 1.0, 10.0, 100.0] # initialise learning rates
    acc_graphs = {} # dictionary of {lr: accuracy array, ...}
    for lr in learning_rates: # runs training and testing over each learning rate
        params = init_weights_and_bias(NIn, NHid, NOut)
        max_test_acc, accuracy_array = train(xtr, ytr, xte, yte, params, batch_size, lr, epochs)
        acc_graphs[lr] = accuracy_array
        print(f"Max Accuracy for LR={lr}: {max_test_acc}") # print max accuracy for each lr
    # Plotting
    x = [i for i in range(epochs)]
    for lr, accs in acc_graphs.items(): # plots curves ontop of eachother
        y = [i*100 for i in accs]
        plt.plot(x, y, marker='o', label=f"lr={lr}")
        plt.xlabel("Epochs")
    plt.ylabel("Accuracy (%)")
```

```
    plt.title("Neural Network Learning Curve")
    plt.grid(True) # adds a grid
    plt.legend()
    plt.show() # display window

def task3(xtr, ytr, xte, yte, NIn, NHid, NOut, learning_rate, epochs): # runs and
plots experiment 3
    batch_sizes = [1, 5, 20, 100, 300] # initialises batch sizes
    max_accs = []
    for bs in batch_sizes: # runs train and test for each batch size
        params = init_weights_and_bias(NIn, NHid, NOut)
        max_test_acc, accuracy_array = train(xtr, ytr, xte, yte, params, bs,
learning_rate, epochs)
        max_accs.append(max_test_acc)
        print(f"Max Accuracy for Batch Size={bs}: {max_test_acc}") # retrive max
accuracy for each batch size
    y = [i*100 for i in max_accs]
    plt.plot(batch_sizes, y, marker='o') # plots max acc vs batch size, does not
use accuracy array
    plt.xlabel("Batch Size")
    plt.ylabel("Accuracy (%)")
    plt.title("Neural Network Learning Curve")
    plt.grid(True) # adds a grid
    plt.show() # display window


def main():
    # parses the arguments
    args = parse_args()
    # Loads the train File
    x_train, y_train = loadFile(args.train_path)
    x_test, y_test = loadFile(args.test_path)
    # Normalise the pixel values
    xtr = normaliseX(x_train)
    xte = normaliseX(x_test)
    # Normalise the labels
    ytr = normaliseY(y_train)
    yte = normaliseY(y_test)
    # inititalise all weights and biases
    params = init_weights_and_bias(args.NInput, args.NHidden, args.NOutput)
    # Task 1
    task1(xtr, ytr, xte, yte, params, batch_size=20, learning_rate=3.0,
epochs=30)
    # Task 2
    task2(xtr, ytr, xte, yte, args.NInput, args.NHidden, args.NOutput,
batch_size=20, epochs=30)
    # Task 3
    task3(xtr, ytr, xte, yte, args.NInput, args.NHidden, args.NOutput,
learning_rate=3.0, epochs=30)
    # Task 4
    params = init_weights_and_bias(args.NInput, args.NHidden, args.NOutput)
```

```
    task1(xtr, ytr, xte, yte, params, batch_size=100, learning_rate=1.0,
epochs=100)



main()

# RUN: python nn.py 784 30 10 fashion-mnist_train.csv.gz fashion-
mnist_test.csv.gz
```